TECHNICAL SYSTEM ARCHITECTURE SPECIFICATION

High-Level Design: Proactive AI Companion Gateway

Project Name:	Project Vigil	**Target Agent:**	Gemini Coding Agent
Architecture:	Self-Hosted / OpenClaw Paradigm	**Date:**	June 2026

1. System Overview

Project Vigil is a self-hosted personal AI assistant runtime modeled after stateful automation systems like OpenClaw. Unlike typical reactive chatbots that only respond to incoming user webhooks, Vigil runs permanently on local, user-managed hardware and utilizes an autonomous background **Proactivity Engine**.

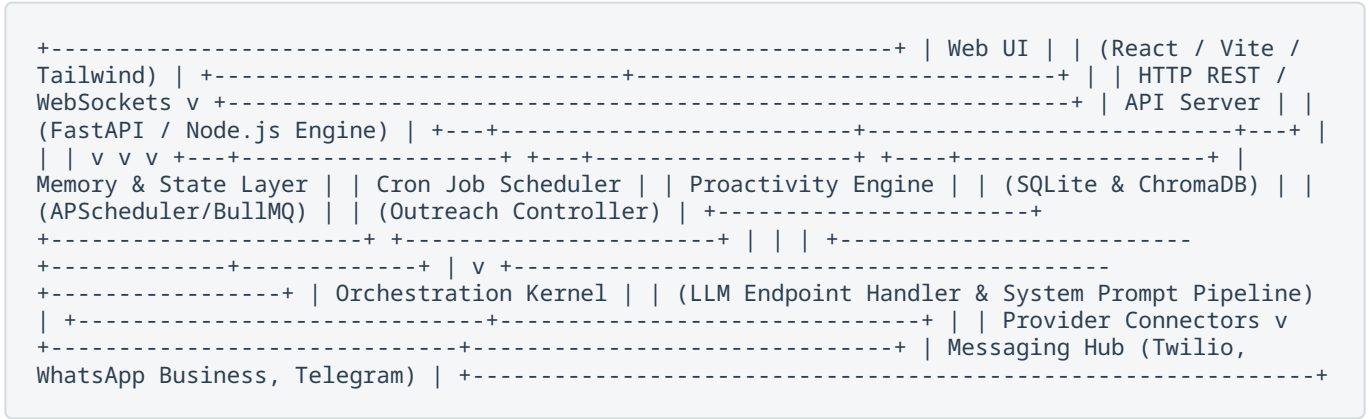
The primary design goal is to grant local AI instances the operational capacity to independently initiate outbound conversations with the user across diverse day-to-day messaging channels (SMS, WhatsApp, Telegram, or Matrix). The system features an integrated, lightweight control-plane WebUI allowing configuration adjustments, prompt variations engineering, pipeline logs, and session management.

Core Constraints & Guarantees

Privacy First: Conversation history, context graphs, vector databases, and keys must remain localized on user hardware.
Hybrid Inference: Native interfaces map directly to local inference backends (Ollama, vLLM) via OpenAI-compatible endpoints, while supporting cloud provider APIs interchangeably.
Asynchronous Resilience: Network drops, channel rate-limiting, or extended LLM compilation times must not freeze runtime orchestration loops.

2. System Topology & Architecture

The following abstract component model outlines the visual block configuration for the system layers, showcasing how the orchestration kernel handles background scheduler cycles and inbound API callbacks.



3. Component Breakdown

3.1. Orchestration Kernel & LLM Client

Acts as the intelligent engine room of the system. Rather than implementing distinct SDK interfaces for every distinct model, the system standardizes communication layers entirely over an OpenAI-compatible spec interface.

- **State Augmentation:** Dynamically stitches together sliding short-term conversation context, user state metadata, time coordinates, and injected factual blocks from long-term memory.
- **Streaming Execution:** Employs async token streaming natively to capture responses transparently and feed internal filters before transmission.

3.2. Messaging Hub (Multi-Channel Router)

The Messaging Hub normalizes disparate third-party structures into structured standard payloads.

- **Abstract Core:** Defines a base structural model class (`BaseMessagingProvider`) enforcing standardized methods like `send_message(user_id, text)` and `on_message_received(payload)`.
- **Webhook Receivers:** Exposes isolated callback ports to capture incoming messages from external gateways asynchronously.
- **Decoupled Queueing:** Incoming interaction packages must execute out-of-band via internal queue processing threads. This protects upstream webhooks from timing out if local models experience high context load latency.

3.3. Proactivity Engine & Cron Scheduler

This engine facilitates autonomous outbound interaction loops without requiring a user-initiated request context.

- **Time-Based Invocations:** Employs structured cron tables to invoke focused "Routine Evaluation Prompts" (e.g., Morning strategic briefings or targeted evening summary processing).
- **Event-Driven Triggers:** Evaluates external environmental states (e.g., system monitors, personal calendars, or web-scraping hooks) to trigger opportunistic outreach scenarios.
- **Guardrails:** Enforces dynamic rate-limiting states and "Do Not Disturb" rules directly against the core storage tracking matrix to prevent logical edge loops from spamming users.

3.4. Storage & Vector Memory Layer

- **Relational Context Engine:** SQLite configuration tracks dynamic system properties, job structures, messaging audit trails, and transactional interaction context histories.
- **Long-Term Vector Retrieval:** Employs simple semantic processing (e.g., localized ChromaDB or PGVector instances) to store user history extracts, preferences, and long-term background knowledge.

3.5. WebUI Control Plane

A unified workspace focused on runtime management, system parameter configurations, and observation metrics.

- **Dashboard Analytics:** Surface processing latencies, model heartbeat status, active messenger integrations, and immediate logs.
- **Credentials & State:** Secure frontend key/token management for communication backends and active API targets.
- **Persona Customizer:** Provides granular prompt tuning interfaces and custom DND period scheduling matrices.

4. Execution Sequences & Data Flows

4.1. Inbound Conversation Routing Flow

```
[User Device] ----(Message Sent)----> [Messaging Provider API]
                                                  |
                                          (HTTPS Post Callback)
                                                  v
[Orchestration Queue] <---(Enqueues Job)--- [Webhook Receiver]
          |
   (Worker Picks Up)
          v
[SQLite / Vector State] ----(Extract History & RAG Context)----+
                                                               |
[Local Model Endpoint] <----(Generate Response Payload)---------+
          |
    (Token Finish)
          v
[Messaging Hub Router] ----(Execute Transmit)----> [User Gateway API]
```

4.2. Autonomous Proactive Pipeline Flow

```
[Scheduler Milestone Trigger]
              |
              v
[Proactivity Engine] ----(Check Restrictions & DND Matrix)
              |
      (Checks Cleared)
              v
[Orchestration Layer] ---(Inject Proactive Evaluation Prompt) ---> [Local Model]
                                                                        |
                                                                (Response Generated)
                                                                        v
[User Device] <----(Dispatched Message Outbound)---- [Messaging Hub Provider]
```

5. Implementation Blueprints (Prompts for Coding Agents)

Provide these steps to your AI coding assistant incrementally to ensure modular construction.

Step 1: Core Base & Messaging Abstraction

```
Context: You are building an open-source proactive AI companion similar to OpenClaw.
Task: Write the core messaging router interface and data structures in Python/TypeScript.
Define an abstract class or interface `BaseMessagingProvider` containing `send_message`
and standard payload extractors for inbound messages. Implement a mock provider
and one real example provider (such as Twilio SMS or Telegram bot).
Ensure asynchronous processing.
```

Step 2: Storage Architecture & Context Management

```
Context: We have the messaging abstraction layer ready. Now we need memory infrastructure.
Task: Implement an SQLite schema using an ORM (SQLAlchemy or Prisma) to track:
- Conversations (id, channel, user_id, sender_type, text, timestamp)
- Configuration (key, value)
- ProactivityLogs (id, execution_time, reason_code, message_dispatched)

Write a repository layer to save messages and fetch a sliding-window chat history
filtered by user and channel coordinates.
```

Step 3: Proactivity Engine Implementation

```
Context: The core database and messaging routes are fully operational.
Task: Write a Proactivity Engine module. It must include a background loop or cron worker
setup.
It needs to check a "Do Not Disturb" parameter in the DB before executing any outbound
calls.
Write a function `trigger_proactive_outreach(reason_code)` that creates an LLM payload,
captures the response, writes the action to `ProactivityLogs`, and dispatches it via
our active Messaging Provider.
```

Step 4: Control Plane Gateway WebUI

> Context: The backend can send, receive, and self-trigger. We require the final administration interface.
> Task: Create a single-page management WebUI using React/Vite with Tailwind CSS.
> It should communicate with a new set of administration backend API routes. The UI needs:
> 1) A system health overview dashboard toggle.
> 2) An environment variable/API key config editor form.
> 3) A real-time system log stream viewer to monitor when the proactivity engine executes.